

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Алгоритм построения наилучшего приближения параболы и гиперболы»

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Алгоритм построения наилучшего приближения параболы и гиперболы	5
2.1 Постановка задачи	5
2.2 Параметрическое представление кривых	5
2.2.1 Параметрическое представление параболы	5
2.2.2 Параметрическое представление гиперболы	7
2.3 План построения наилучшего приближения	9
3 Описание реализации построения наилучшего приближения	10
3.1 Построение наилучшего приближения параболы	10
3.1.1 Ручная реализация	11
3.1.2 Библиотечная реализация	12
3.2 Построение наилучшего приближения гиперболы	14
3.2.1 Ручная реализация	14
3.2.2 Библиотечная реализация	15
4 Результаты работы	18
5 Сравнение собственной реализации с библиотечной	20
Заключение	22

Введение

В современной вычислительной математике и компьютерной графике ключевую роль играет точное представление геометрических объектов. Кривые второго порядка, в частности парабола и гипербола, являются фундаментальными элементами при моделировании физических процессов, проектировании механизмов и создании цифровых контуров.

Основная проблема при дискретизации кривых заключается в том, что равномерное изменение параметра t не гарантирует равномерного распределения точек вдоль длины дуги. В областях с высокой кривизной или скоростью изменения координат точки могут сгущаться или разрежаться, что приводит к неравномерности аппроксимации. Поэтому задачей построения наилучшего приближения является параметризация кривой по длине дуги, обеспечивающая геометрически равномерное распределение узлов. Это обеспечивает точную визуализацию и повышение точности численных расчётов.

Сравнение различных подходов в реализации алгоритма позволяет понять, какой из них эффективнее при конкретных начальных условиях.

1 Постановка задачи

Цель лабораторной работы: реализация алгоритма построения наилучшего приближения параболы и гиперболы.

В рамках поставленной цели необходимо выполнить следующие задачи:

1. Изучить алгоритмы построения наилучшего приближения параболы и гиперболы.
2. Реализовать алгоритмы построения наилучшего приближения параболы и гиперболы вручную, используя язык C++.
3. Реализовать те же алгоритмы с использованием специализированных библиотек языка C++.
4. Провести сравнение производительности собственной и библиотечной реализации и визуализировать результаты построения гиперболы и параболы.

2 Алгоритм построения наилучшего приближения параболы и гиперболы

2.1 Постановка задачи

Дано: уравнение кривой, n — количество точек, которые хотим выдать, $[x_{\min}, x_{\max}]$ для гиперболы/параболы.

Надо: вычислить координаты (x_i, y_i) , $i = 1, \dots, n$, с минимальной погрешностью, равномерно расставленные по кривой.

2.2 Параметрическое представление кривых

В параметрическом виде каждая координата точки кривой представлена как функция одного параметра. Значение параметра задаёт координатный вектор точки на кривой. Для двумерной кривой с параметром t координаты точки равны: $x = x(t)$, $y = y(t)$.

Тогда векторное представление точки на кривой:

$$\mathbf{P}(t) = [x(t) \quad y(t)] .$$

Параметрическая форма позволяет представить замкнутые и многозначные кривые. Производная, то есть касательный вектор, есть

$$\mathbf{P}'(t) = [x'(t) \quad y'(t)] ,$$

где $'$ обозначает дифференцирование по параметру. Наклон кривой dy/dx равен

$$\frac{dy}{dx} = \frac{dy/dt}{dx/dt} = \frac{y'(t)}{x'(t)} .$$

Так как точка на параметрической кривой определяется только значением параметра, эта форма не зависит от выбора системы координат. Конечные точки и длина кривой определяются диапазоном изменения параметра. Часто бывает удобно нормализовать параметр на интересующем отрезке кривой к $0 \leq t \leq 1$.

2.2.1 Параметрическое представление параболы

Рассмотрим параболу с вершиной в центре координат, раскрытую вправо, т. е. с осью симметрии — положительной полуосью x . На рисунке 1 изображена верхняя ветвь такой параболы.

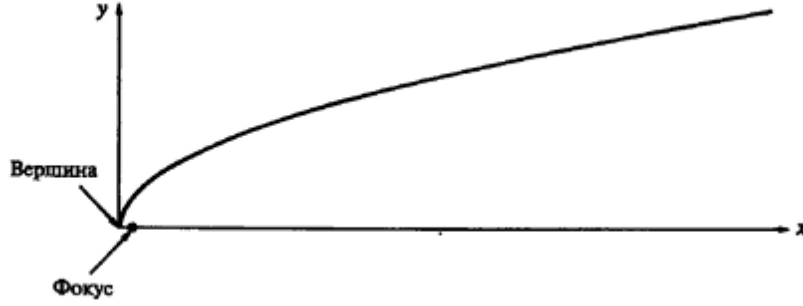


Рис. 1: Парабола

В прямоугольных координатах непараметрическое представление параболы:

$$y^2 = 4ax \quad (1)$$

Параметрическое представление имеет вид:

$$x = a\theta^2, \quad y = 2a\theta, \quad \text{где } 0 \leq \theta \leq \pi/2. \quad (2)$$

Хотя оно обеспечивает достаточно хорошее изображение, получаемая фигура не является фигурой с максимальной вписанной площадью, и поэтому это не оптимальный вариант. Другое параметрическое представление действительно даёт фигуру с наибольшей вписанной площадью:

$$x = a\theta^2, \quad y = 2a\theta, \quad (3)$$

где $0 < \theta < \infty$ соответствует всей верхней ветви параболы. В отличие от эллипса парабола не замкнутая кривая, поэтому изображаемая часть должна быть ограничена минимальным и максимальным значением параметра. Это можно сделать несколькими способами. Если диапазон изменения координаты x ограничен, то

$$\theta_{\min} = \sqrt{\frac{x_{\min}}{a}}, \quad \theta_{\max} = \sqrt{\frac{x_{\max}}{a}} \quad (4)$$

Если ограничен диапазон изменения y , то

$$\theta_{\min} = \frac{y_{\min}}{2a}, \quad \theta_{\max} = \frac{y_{\max}}{2a} \quad (5)$$

Установив $\theta_{\min}$ и/или $\theta_{\max}$, можно построить параболу в первом квадранте. Параболы в других квадрантах, со смещённым центром и в других ориентациях строятся с помощью отражения, поворота и переноса.

Параболу можно построить также, пользуясь приращениями параметра. Пусть на параболе задано фиксированное количество точек, т. е. приращение параметра θ постоянно. Из $\theta_{i+1} = \theta_i + \Delta\theta$ уравнение (3) принимает вид:

$$x_{i+1} = a(\theta_i + \Delta\theta)^2 = a\theta_i^2 + 2a\theta_i\Delta\theta + a(\Delta\theta)^2$$

$$y_{i+1} = 2a(\theta_i + \Delta\theta) = 2a\theta_i + 2a\Delta\theta$$

Используя уравнение (3) с $\theta = 0$, перепишем формулы:

$$x_{i+1} = x_i + y_i\Delta\theta + a(\Delta\theta)^2 \quad (6)$$

$$y_{i+1} = y_i + 2a\Delta\theta \quad (7)$$

Расчёт очередной точки требует трёх сложений и одного умножения во внутреннем цикле алгоритма. На рисунке 2 приведён пример параболы, сгенерированной по рекурсивным формулам (6)–(7).

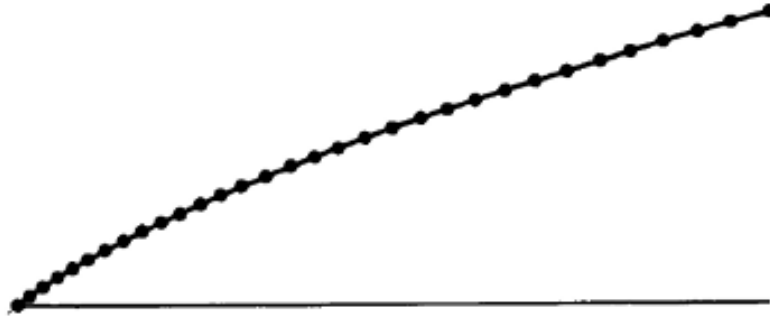


Рис. 2: Параметрически сгенерированная парабола

2.2.2 Параметрическое представление гиперболы

Построим гиперболу с центром в начале координат и осью симметрии, совпадающей с осью x . Её непараметрическое представление в прямоугольных координатах:

$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$$

При этом вершина находится в точке $(a, 0)$, оси асимптот $\pm b/a$. Вид параметрического представления:

$$x = \pm a \sec \theta, \quad y = \pm b \tan \theta \quad (8)$$

где $0 \leq \theta < \pi/2$, даёт искомую гиперболу. Для такого представления площадь вписанного многоугольника не максимальна. Однако она близка к максимальной, и с помощью формулы суммы углов можно получить эффективный алгоритм. Вспомним, что

$$\sec(\theta + \Delta\theta) = \frac{1}{\cos(\theta + \Delta\theta)} = \frac{1}{\cos \theta \cos \Delta\theta - \sin \theta \sin \Delta\theta}$$

$$\tan(\theta + \Delta\theta) = \frac{\tan \theta + \tan \Delta\theta}{1 - \tan \theta \tan \Delta\theta}$$

Подставим в уравнения (8):

$$x_{i+1} = \pm \frac{ab}{b \cos \Delta\theta - y_i \sin \Delta\theta}$$

$$y_{i+1} = \pm \frac{b(y_i + b \tan \Delta\theta)}{b - y_i \tan \Delta\theta}$$

Используя уравнения (8) с $\theta = 0$, перепишем эти уравнения как:

$$x_{i+1} = \pm \frac{ab}{b \cos \Delta\theta - y_i \sin \Delta\theta} \quad (9)$$

$$y_{i+1} = \pm \frac{b(y_i + b \tan \Delta\theta)}{b - y_i \tan \Delta\theta} \quad (10)$$

Другое параметрическое представление гиперболы, дающее максимальную вписанную площадь:

$$x = a \cosh \theta, \quad y = b \sinh \theta \quad (11)$$

Гиперболические функции определяются как $\cosh \theta = (e^\theta + e^{-\theta})/2$ и $\sinh \theta = (e^\theta - e^{-\theta})/2$. При изменении θ от 0 до бесконечности проходит вся гипербола. Формула для суммы углов для них:

$$\cosh(\theta + \Delta\theta) = \cosh \theta \cosh \Delta\theta + \sinh \theta \sinh \Delta\theta$$

$$\sinh(\theta + \Delta\theta) = \sinh \theta \cosh \Delta\theta + \cosh \theta \sinh \Delta\theta$$

Это позволяет записать уравнения (11) как:

$$x_{i+1} = a(\cosh \theta \cosh \Delta\theta + \sinh \theta \sinh \Delta\theta)$$

$$y_{i+1} = b(\sinh \theta \cosh \Delta\theta + \cosh \theta \sinh \Delta\theta)$$

Или:

$$x_{i+1} = x_i \cosh \Delta\theta + \frac{a}{b} y_i \sinh \Delta\theta \quad (12)$$

$$y_{i+1} = \frac{b}{a} x_i \sinh \Delta\theta + y_i \cosh \Delta\theta \quad (13)$$

Чтобы ограничить область гиперболы, необходимо установить минимальное и максимальное значения. Пусть ветвь гиперболы лежит в первом и четвёртом квадранте и рассматривается часть при $x_{\min} \leq x \leq x_{\max}$. Тогда

$$\theta_{\min} = \operatorname{arch} \frac{x_{\min}}{a}, \quad \theta_{\max} = \operatorname{arch} \frac{x_{\max}}{a} \quad (14)$$

где обратный гиперболический косинус получен как:

$$\operatorname{arch}(x) = \ln(x + \sqrt{x^2 - 1}) \quad (15)$$

Остальные границы определяются аналогично. Пример части гиперболы в первом квадранте, полученного этим методом, показан на рисунке 3.

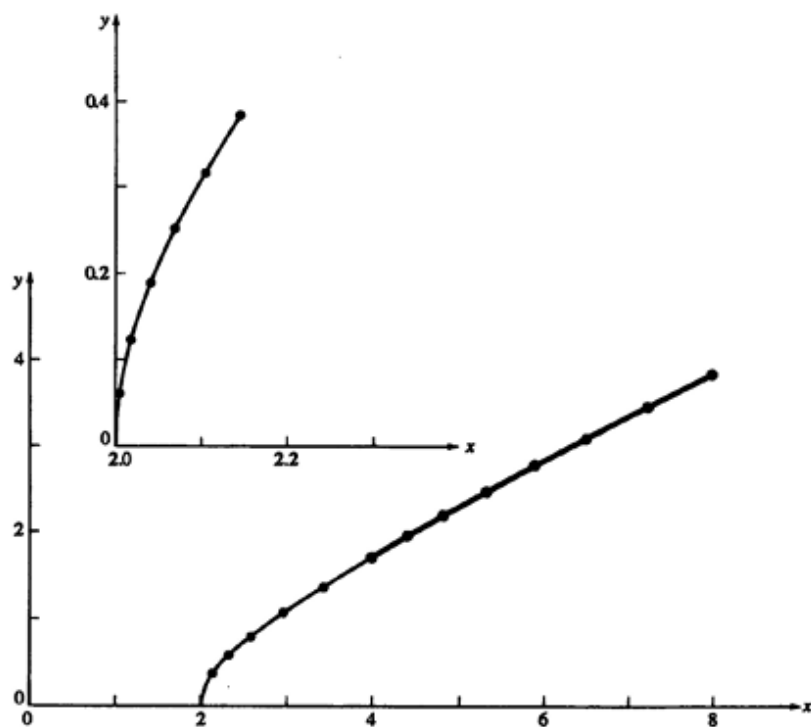


Рис. 3: Параметрическая гипербола

2.3 План построения наилучшего приближения

1. Строим параметрическое представление $x(t)$, $y(t)$.
2. Дискретизируем параметр $dt \rightarrow \Delta t$. (Точки должны быть равномерно распределены по длине кривой, а не по параметру t . Это требует учёта скорости изменения длины кривой относительно параметра t .)
3. Строим рекуррентную формулу соответственно типу кривой (координаты следующих точек на основе старых).

3 Описание реализации построения наилучшего приближения

Во всех реализациях используется один и тот же принцип: точки кривой должны быть равномерно распределены по длине дуги, а не по параметру t . Это требует перехода от параметрического задания кривой к параметризации по длине дуги.

Общий алгоритм:

1. Задаётся параметрическое представление кривой: $x = x(t)$, $y = y(t)$.
2. Вычисляется скорость изменения длины дуги:

$$v(t) = \frac{ds}{dt} = \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2}$$

3. Численно интегрируется длина дуги:

$$s(t) = \int_0^t v(\tau) d\tau$$

4. Строится равномерная сетка по длине дуги:

$$s_i = \frac{i}{n-1} s_{\max}, \quad i = 0, \dots, n-1$$

5. Численно находится обратная зависимость $t = t(s)$.
6. Вычисляются координаты точек: $x_i = x(t_i)$, $y_i = y(t_i)$.

Различия между параболой и гиперболой в алгоритме заключаются в формулах скорости и координат.

3.1 Построение наилучшего приближения параболы

Рассматривается параметрически заданная парабола:

$$x(\theta) = a \tan^2 \theta, \quad y(\theta) = 2a \tan \theta, \quad \theta \in [\theta_{\min}, \theta_{\max}), \quad 0 \leq \theta < \frac{\pi}{2}. \quad (16)$$

Скорость изменения длины дуги:

$$\begin{aligned} v(\theta) &= \sqrt{(x'(\theta))^2 + (y'(\theta))^2} = \sqrt{(2a \tan \theta \sec^2 \theta)^2 + (2a \sec^2 \theta)^2} \\ &= 2a \sec^3 \theta \sqrt{\tan^2 \theta + 1} = 2a \sec^3 \theta. \end{aligned}$$

Задача построения наилучшего приближения — получить точки (x_i, y_i) , равномерно распределённые по длине дуги.

3.1.1 Ручная реализация

Функция `manual_parabola` строит наилучшее приближение параболы с помощью рекуррентных формул, основанных на формулах сложения тригонометрических функций.

Параметры:

- n — количество точек на кривой;
- a — коэффициент параболы;
- `t_max` — максимальное значение параметра θ .

Вспомогательные переменные. Введём вместо $\tan \theta$ пару вспомогательных переменных:

$$p_i = \sin \theta_i, \quad q_i = \cos \theta_i, \quad (17)$$

тогда $\tan \theta_i = p_i/q_i$ и координаты точки выражаются как:

$$x_i = a \left(\frac{p_i}{q_i} \right)^2, \quad y_i = \frac{2a p_i}{q_i}. \quad (18)$$

Рекуррентные формулы. При шаге θ_{i+1} применим формулы сложения тригонометрических функций:

$$\sin(\theta_i + \Delta\theta) = \sin \theta_i \cos \Delta\theta + \cos \theta_i \sin \Delta\theta, \quad (19)$$

$$\cos(\theta_i + \Delta\theta) = \cos \theta_i \cos \Delta\theta - \sin \theta_i \sin \Delta\theta. \quad (20)$$

Инициализация. Из $\tan \theta = \sqrt{x/a}$ получаем:

$$\theta_{\min} = \arctan \sqrt{\frac{x_{\min}}{a}}, \quad \theta_{\max} = \arctan \sqrt{\frac{x_{\max}}{a}}, \quad (21)$$

при $x_{\min} = 0$: $\theta_{\min} = 0$, $p_0 = 0$, $q_0 = 1$. Шаг параметра $\Delta\theta = (\theta_{\max} - \theta_{\min})/(n - 1)$.

Листинг 1: Функция `manual_parabola`

```
1 #include <vector>
2 #include <cmath>
3 #include <utility>
4
5 std::pair<std::vector<double>, std::vector<double>>
6 manual_parabola(int n, double a = 1.0, double t_max = 2.0)
7 {
8     // Диапазон параметра theta через arctan
9     double x_max = a * t_max * t_max;
10    double theta_min = std::atan(std::sqrt(0.0)); // = 0
```

```

11     double theta_max = std::atan(std::sqrt(x_max / a)); // = atan
        (t_max)
12     double dtheta      = (n > 1) ? (theta_max - theta_min) / (n -
        1) : 0.0;
13
14     double c = std::cos(dtheta);
15     double s = std::sin(dtheta);
16
17     std::vector<double> x(n), y(n);
18
19     // Начальные значения sin и cos
20     double p = std::sin(theta_min); // sin(theta_0) = 0
21     double q = std::cos(theta_min); // cos(theta_0) = 1
22
23     for (int i = 0; i < n; ++i) {
24         // Координаты из tan(theta) = p / q
25         double tan_theta = p / q;
26         x[i] = a * tan_theta * tan_theta;
27         y[i] = 2.0 * a * tan_theta;
28
29         double p_next = p * c + q * s;
30         double q_next = q * c - p * s;
31         p = p_next;
32         q = q_next;
33     }
34
35     return {x, y};
36 }

```

3.1.2 Библиотечная реализация

Функция `library_parabola` предназначена для построения наилучшего приближения параболы с использованием стандартных библиотек языка C++. Функция возвращает массивы координат x , y , представляющих точки параболы.

Дискретизация параметра и вычисление скорости происходит с помощью стандартных средств `<cmath>` и `<vector>`. Для получения длины дуги $s(t)$ используется метод трапеций, реализованный вручную средствами стандартной библиотеки `<numeric>`. Обратная функция $t = t(s)$ строится с помощью `std::lower_bound` из `<algorithm>` и линейной интерполяции. Далее строится равномерная сетка $s_i = \frac{i}{n-1}L$. Функция получает параметры и по ним вычисляет координаты, используя формулы для параболы:

$$x(t) = t, \quad y(t) = at^2, \quad t \in [0, t_{\max}].$$

Листинг 2: Функция `library_parabola`

```

1 #include <vector>
2 #include <cmath>

```

```

3 #include <algorithm>
4 #include <numeric>
5 #include <utility>
6
7 std::pair<std::vector<double>, std::vector<double>>
8 library_parabola(int n, double a = 1.0, double t_max = 2.0)
9 {
10     const int N = 10000;
11
12     // Равномерная сетка по параметру
13     std::vector<double> t(N), speed(N), s(N);
14     for (int i = 0; i < N; ++i)
15         t[i] = t_max * i / (N - 1);
16
17     // Скорость изменения длины дуги:  $v(t) = \sqrt{1 + (2at)^2}$ 
18     for (int i = 0; i < N; ++i)
19         speed[i] = std::sqrt(1.0 + std::pow(2.0 * a * t[i], 2));
20
21     // Кумулятивное интегрирование методом трапеций
22     s[0] = 0.0;
23     for (int i = 1; i < N; ++i)
24         s[i] = s[i - 1] + (speed[i - 1] + speed[i]) * (t[i] - t[i - 1]) / 2.0;
25
26     // Равномерная сетка по длине дуги
27     std::vector<double> s_uniform(n), t_uniform(n);
28     for (int i = 0; i < n; ++i)
29         s_uniform[i] = s[N - 1] * i / (n - 1);
30
31     // Линейная интерполяция: обратная функция  $t = t(s)$ 
32     for (int i = 0; i < n; ++i) {
33         auto it = std::lower_bound(s.begin(), s.end(), s_uniform[i]);
34         if (it == s.end()) {
35             t_uniform[i] = t[N - 1];
36         } else if (it == s.begin()) {
37             t_uniform[i] = t[0];
38         } else {
39             int idx = static_cast<int>(std::distance(s.begin(), it));
40             double frac = (s_uniform[i] - s[idx - 1]) / (s[idx] - s[idx - 1]);
41             t_uniform[i] = t[idx - 1] + frac * (t[idx] - t[idx - 1]);
42         }
43     }
44
45     // Вычисление координат
46     std::vector<double> x(n), y(n);
47     for (int i = 0; i < n; ++i) {
48         x[i] = t_uniform[i];
49         y[i] = a * t_uniform[i] * t_uniform[i];

```

```

50     }
51
52     return {x, y};
53 }

```

3.2 Построение наилучшего приближения гиперболы

Рассматривается параметрически заданная гипербола:

$$x(t) = a \cosh t, \quad y(t) = b \sinh t, \quad t \in [0, t_{\max}],$$

скорость изменения длины дуги равна $v(t) = \sqrt{(a \sinh t)^2 + (b \cosh t)^2}$.

Длина дуги:

$$s(t) = \int_0^t \sqrt{(a \sinh \tau)^2 + (b \cosh \tau)^2} d\tau$$

Задача наилучшего приближения заключается в построении точек (x_i, y_i) , равномерно распределённых по длине дуги, то есть:

$$s_i = \frac{i}{n-1} L, \quad L = s(t_{\max})$$

где n — количество точек.

3.2.1 Ручная реализация

Функция `manual_hyperbola` предназначена для построения гиперболы вручную. Функция возвращает массивы координат x, y , представляющие точки гиперболы.

Параметры:

- `n` — количество точек на кривой;
- `a, b` — коэффициенты гиперболы по осям X и Y соответственно;
- `t_max` — максимальное значение параметра t (соответствует параметру θ).

Шаг $\Delta\theta$ определяется как $\Delta\theta = (\theta_{\max} - \theta_{\min})/(n-1)$. Предварительно вычисляются константы $\cosh \Delta\theta$ и $\sinh \Delta\theta$. Первая точка кривой вычисляется напрямую по уравнениям гиперболы, а для каждой последующей точки $i = 1, \dots, n-1$ координаты обновляются по рекуррентным формулам:

$$x_i = x_{i-1} \cosh \Delta\theta + \frac{a}{b} y_{i-1} \sinh \Delta\theta$$

$$y_i = y_{i-1} \cosh \Delta\theta + \frac{b}{a} x_{i-1} \sinh \Delta\theta$$

Листинг 3: Функция manual_hyperbola

```

1  #include <vector>
2  #include <cmath>
3  #include <utility>
4
5  std::pair<std::vector<double>, std::vector<double>>
6  manual_hyperbola(int n, double a = 1.0, double b = 1.0, double
7      t_max = 2.0)
8  {
9      double theta_min = 0.0;
10     double theta_max = t_max;
11     double delta_theta = (n > 1) ? (theta_max - theta_min) / (n -
12         1) : 0.0;
13
14     // Предвычисление констант
15     double cosh_dtheta = std::cosh(delta_theta);
16     double sinh_dtheta = std::sinh(delta_theta);
17
18     std::vector<double> x(n), y(n);
19
20     // Первая точка
21     x[0] = a * std::cosh(theta_min);
22     y[0] = b * std::sinh(theta_min);
23
24     // Рекуррентный цикл
25     for (int i = 1; i < n; ++i) {
26         double xprev = x[i - 1];
27         double yprev = y[i - 1];
28         x[i] = xprev * cosh_dtheta + (a / b) * yprev *
29             sinh_dtheta;
30         y[i] = (b / a) * xprev * sinh_dtheta + yprev *
31             cosh_dtheta;
32     }
33
34     return {x, y};
35 }

```

3.2.2 Библиотечная реализация

Функция `library_hyperbola` предназначена для построения наилучшего приближения гиперболы с использованием стандартных библиотек языка C++. Функция возвращает массивы координат x , y , представляющих точки гиперболы.

Дискретизация параметра и вычисление скорости происходит с помощью функций `std::cosh`, `std::sinh` и `std::sqrt` из заголовочного файла `<cmath>`. Функция создаёт массив из 10000 равномерно распределённых точек на отрезке $[0, t_{\max}]$. Для получения длины дуги $s(t)$ используется метод трапеций, реализованный средствами стандартной библиотеки. Обратная функция строится с помощью `std::lower_bound` из `<algorithm>` и линейной

интерполяции. Далее строится равномерная сетка $s_i = \frac{i}{n-1}L$. Функция вычисляет координаты, используя формулы для гиперболы:

$$x(t) = a \cosh t, \quad y(t) = b \sinh t, \quad t \in [0, t_{\max}].$$

Листинг 4: Функция library_hyperbola

```

1  #include <vector>
2  #include <cmath>
3  #include <algorithm>
4  #include <utility>
5
6  std::pair<std::vector<double>, std::vector<double>>
7  library_hyperbola(int n, double a = 1.0, double b = 1.0, double
   t_max = 2.0)
8  {
9      const int N = 10000;
10
11     // Равномерная сетка по параметру
12     std::vector<double> t(N), speed(N), s(N);
13     for (int i = 0; i < N; ++i)
14         t[i] = t_max * i / (N - 1);
15
16     // Скорость:  $v(t) = \sqrt{(a \sinh(t))^2 + (b \cosh(t))^2}$ 
17     for (int i = 0; i < N; ++i) {
18         double sh = a * std::sinh(t[i]);
19         double ch = b * std::cosh(t[i]);
20         speed[i] = std::sqrt(sh * sh + ch * ch);
21     }
22
23     // Кумулятивное интегрирование методом трапеций
24     s[0] = 0.0;
25     for (int i = 1; i < N; ++i)
26         s[i] = s[i - 1] + (speed[i - 1] + speed[i]) * (t[i] - t[i - 1]) / 2.0;
27
28     // Равномерная сетка по длине дуги
29     std::vector<double> s_uniform(n), t_uniform(n);
30     for (int i = 0; i < n; ++i)
31         s_uniform[i] = s[N - 1] * i / (n - 1);
32
33     // Линейная интерполяция: обратная функция  $t = t(s)$ 
34     for (int i = 0; i < n; ++i) {
35         auto it = std::lower_bound(s.begin(), s.end(), s_uniform[i]);
36         if (it == s.end()) {
37             t_uniform[i] = t[N - 1];
38         } else if (it == s.begin()) {
39             t_uniform[i] = t[0];
40         } else {
41             int idx = static_cast<int>(std::distance(s.begin(),
   it));

```



```

42         double frac = (s_uniform[i] - s[idx - 1]) / (s[idx] -
43             s[idx - 1]);
44         t_uniform[i] = t[idx - 1] + frac * (t[idx] - t[idx -
45             1]);
46     }
47
48     // Вычисление координат
49     std::vector<double> x(n), y(n);
50     for (int i = 0; i < n; ++i) {
51         x[i] = a * std::cosh(t_uniform[i]);
52         y[i] = b * std::sinh(t_uniform[i]);
53     }
54     return {x, y};
55 }

```

4 Результаты работы

Результат построения наилучшего приближения параболы обоими методами с параметрами $a = 1$, $\Theta = 2$ из 10 точек представлен на рисунке 4.

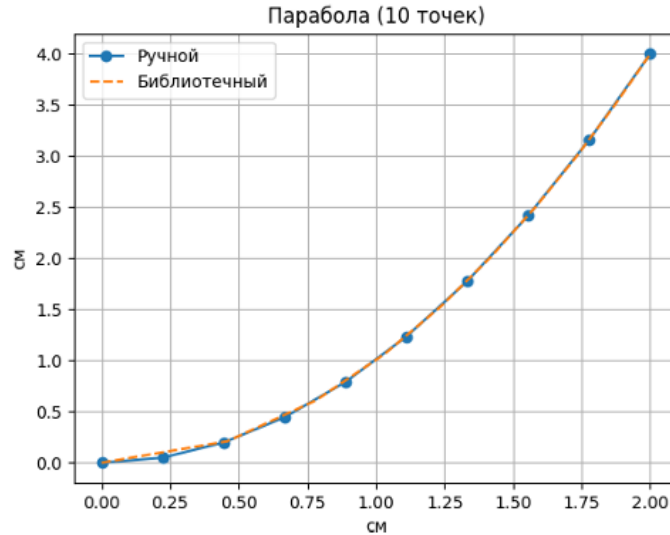


Рис. 4: Построение наилучшего приближения параболы из 10 точек

Результат построения наилучшего приближения параболы обоими методами с параметрами $a = 1$, $\Theta = 2$ из 10000 точек представлен на рисунке 5.

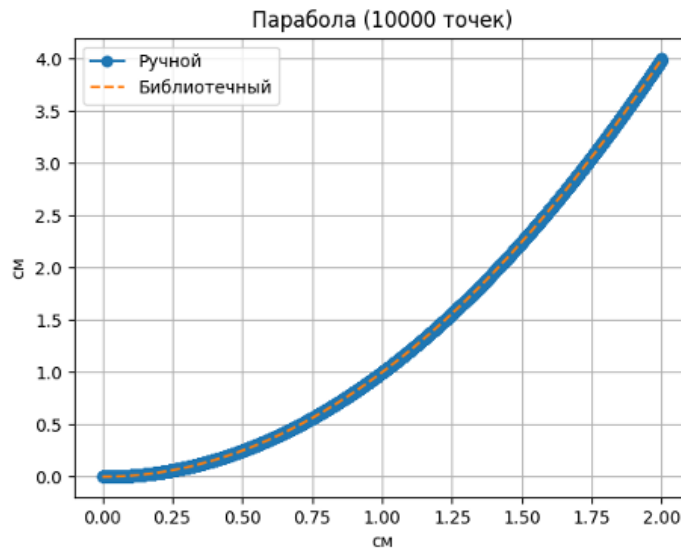


Рис. 5: Построение наилучшего приближения параболы из 10000 точек

Результат построения наилучшего приближения гиперболы с параметрами $a = 1$, $b = 1$, $\Theta = 2$ из 10 точек представлен на рисунке 6.

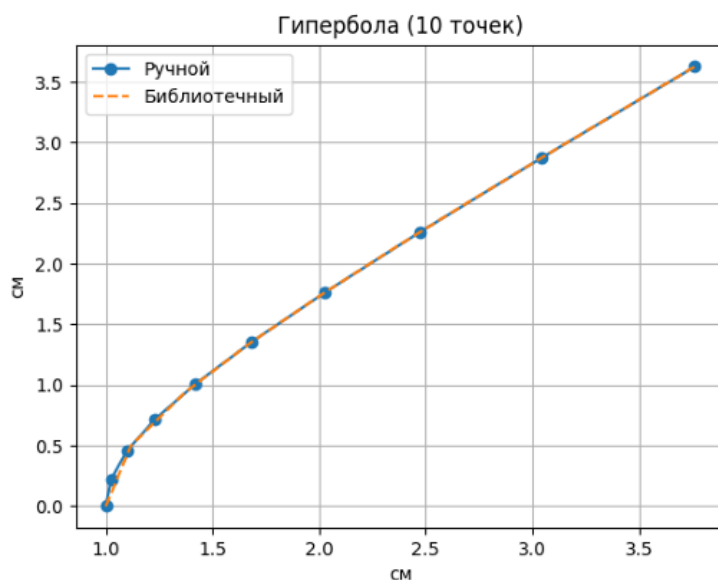


Рис. 6: Построение наилучшего приближения гиперболы из 10 точек

Результат построения наилучшего приближения гиперболы с параметрами $a = 1$, $b = 1$, $\Theta = 2$ из 10000 точек представлен на рисунке 7.

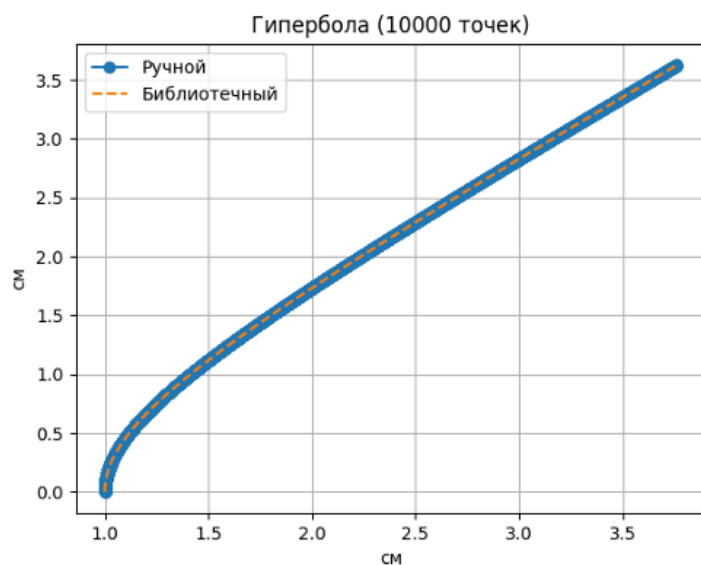


Рис. 7: Построение наилучшего приближения гиперболы из 10000 точек

5 Сравнение собственной реализации с библиотечной

После реализации собственного алгоритма и реализации алгоритма с использованием стандартных библиотек языка C++ (`<cmath>`, `<vector>`, `<algorithm>`, `<numeric>`) был проведён анализ среднего времени построения кривых при различном числе точек в множестве. Для повышения точности выполнялось по 10 измерений каждого числа точек в множестве, а затем их усреднение.

Результаты для построения параболы представлены в таблице 1, для гиперболы — в таблице 2.

Таблица 1: Сравнение среднего времени построения параболы

Количество точек	Собственная реализация	Библиотечная реализация
10	0.000041	0.000249
100	0.000106	0.000262
1000	0.000955	0.000294
10000	0.009367	0.000768

Таблица 2: Сравнение среднего времени построения гиперболы

Количество точек	Собственная реализация	Библиотечная реализация
10	0.000036	0.000443
100	0.000114	0.000487
1000	0.001083	0.000532
10000	0.010557	0.001107

Графики, показывающие разницу времени выполнения собственной реализации алгоритма и реализации с использованием библиотечных функций, показаны на рисунках 8–9.

Такая производительность объясняется различной реализацией: при малых n собственная реализация быстрее, так как у неё меньше накладных расходов, чем у библиотечной. При больших n ситуация меняется: библиотечная функция использует оптимизированные операции над непрерывными блоками памяти стандартного контейнера `std::vector`, тогда как собственная реализация включает цикл сложностью $O(n)$.

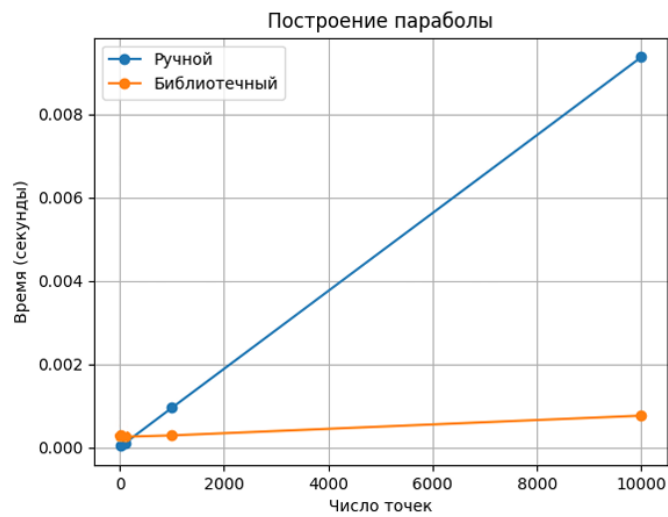


Рис. 8: Сравнение среднего времени построения параболы для различного числа точек

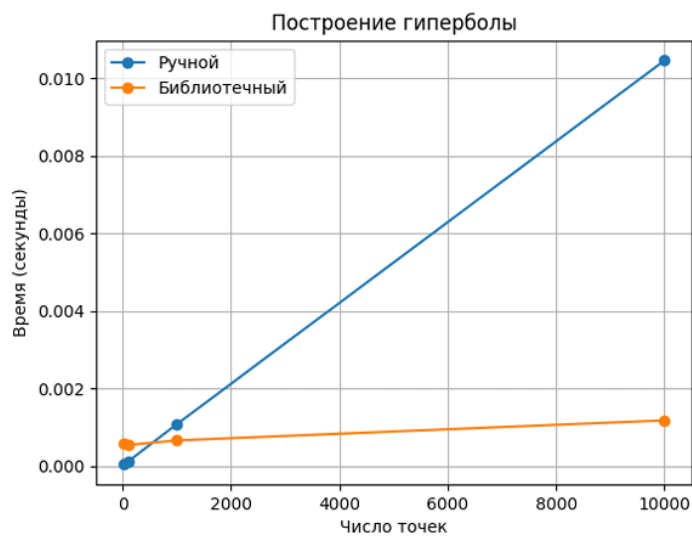


Рис. 9: Сравнение среднего времени построения гиперболы для различного числа точек

Заключение

В ходе данной лабораторной работы мной были изучены алгоритмы построения наилучшего приближения параболы и гиперболы. Каждый из алгоритмов был выполнен двумя способами:

- с использованием функций, написанных в соответствии с математическим описанием алгоритмов (на языке C++);
- с использованием стандартных библиотек языка C++: `<cmath>`, `<vector>`, `<algorithm>`, `<numeric>`.

Каждый алгоритм был запущен на множествах 10, 100, 1000 и 10000 точек.

Для каждого из методов было проведено сравнение скорости построения кривых собственной и библиотечной реализации. В результате было выявлено, что собственные функции наиболее эффективны при малых n ; при n больших 1000 эффективными становятся функции, использующие библиотеки. Это связано с тем, что реализация библиотечных функций использует оптимизированные операции над контейнерами, тогда как у собственной реализации меньше накладных расходов и цикл сложностью $O(n)$.

Таким образом, работа позволила не только изучить математическую модель построения наилучшего приближения параболы и гиперболы, но и провести сравнительный анализ различных способов их программной реализации на языке C++.

Список литературы

- [1] Роджерс, Д.; Адамс, Дж. *Математические основы машинной графики*. М.: Издательство «Мир», 2001. — 555 с. (дата обращения: 10.05.2026).